



Institut Supérieur d'informatique et
de Mathématiques de Monastir



Université de Monastir

Système d'exploitation 2

RIM MEGDICHE
rim.magdich@gmail.com

Chapitre 2

Communication inter-processus

Introduction

Les processus ne sont pas des entités indépendantes. Ils coopèrent souvent pour traiter un même problème ou se mettent en compétition pour utiliser une ressource partagée. Ces processus peuvent s'exécuter en parallèle sur un même ordinateur (monoprocasseur ou multiprocesseurs) ou bien sur des ordinateurs différents. Ils doivent alors s'échanger des informations (**communication interprocessus**).



Pourquoi les processus doivent-ils coopérer ?



Synchronisation

Coordonner l'exécution temporelle.



Échange de données

Communication directe entre processus.



Partage de ressources

Utilisation commune sans interblocage.



Cohérence des données

Partager l'information sans provoquer d'incohérences.

La communication permet à l'exécution d'un processus d'influencer l'exécution d'un autre.

Introduction

Exemple du Compte Bancaire Partagé

- Imagine un compte bancaire avec un solde de 100 €. Deux personnes (**Processus A** et **Processus B**) possèdent une carte pour ce même compte et tentent de retirer de l'argent exactement au même moment.

- 1. **Le code logique** (ce qui devrait se passer):

- Lire le solde actuel.
- Calculer le nouveau solde (Solde - Retrait).
- Écrire le nouveau solde dans la base de données.



- 2. **Le scénario de la catastrophe (Race Condition):**

Temps	Processus A (-20)	Processus B (-50)	Solde en Mémoire
T1	Lit le solde (100€)	100 €	100€
T2	Interruption CPU	Lit le solde 100	100€
T3	Calcule (100 - 20 = 80)	Calcule 100 - 50 = 50	100
T4	Écrit le solde (80€)		80€
T5		50 €	50 €



- 3. **Résultat final** : À la fin, le solde est de 50 €.

- Le problème : On a retiré $20 + 50 = 70$ €. Le solde devrait être de $100 - 70 = 30$ €.
- Pourquoi? Le Processus B a écrasé le travail du Processus A car il a lu le solde avant que A ne le mette à jour. Les 20 € de A ont « disparu » dans la nature (ou plutôt, la banque a fait un cadeau au client). $\rightarrow 100 \text{ €} - \rightarrow 30 \text{ €}$



Introduction

- **Conflit:**

- Si le **Processus A** est interrompu pendant l'exécution de ses instructions et que entre-temps l'ordonnanceur passe la main au **Processus B** puis revient à nouveau au **Processus A** sans que le solde ait été mis à jour.
- Les deux processus retirent donc de l'argent du même compte comme si le solde n'avait pas été changé : bien que les deux retraits soient cohérents, au final un seul retrait a été réellement pris en compte.

Solution : Pour éviter ces conditions de course, il faut empêcher la lecture ou l'écriture de données partagées à plus d'un processus à la fois.



Temps	Processus A (Retrait 20€)	Solde en Mémoire
T1	100€	100€
T2	Interruption CPU	
T3	Calcule ($100 - 20 = 80$)	100€
T4	Écrit le solde (80€)	80€
T5	Écrit le solde (50€)	50€

Exclusion Mutuelle :

- permet d'assurer que si un processus utilise une variable ou ressources partagées, les autres processus seront exclus de la même activité.

Le parallélisme dans les SE

- Nous avons vu que le rôle du SE consistait à répartir des ressources (en particulier le processeur) entre un certain nombre de processus progressant de manière concurrente.
- La présence de tels processus pose un certain nombre de problèmes que l'on peut classer en deux catégories :
 - ✓ les problèmes liés à la **compétition** entre ces processus pour l'obtention des ressources. Il existe des règles d'accès aux ressources qui doivent être respectés. C'est le SE qui règle les conflits lorsqu'ils se présentent, et ce de façon transparente pour les processus en cause.
 - ✓ Les problèmes liés à coopération entre processus contribuant à une tâche globale. Cette coopération qui est prévue dans le code de chacun des processus s'effectue à l'aide de primitives de synchronisation.

Le parallélisme dans les SE

- L'accès par un processus condamne tout accès par les autres (**Exclusion mutuelle**). On qualifie cette situation par l'accès exclusif à une ressource.
- Les exemples de ressources auxquels l'accès est exclusif sont nombreux, nous citons :
 - ✓ certains périphériques : imprimante par exemple.
 - ✓ Une donnée partagée modifiée par des cycles lecture-modification-écriture. (un enregistrement dans un fichier).
 - ✓ Des données du SE manipulées dans les mêmes conditions par des processus systèmes (en particulier, les données susceptibles d'être modifiées, à la fois au niveau principal et sous interruption).

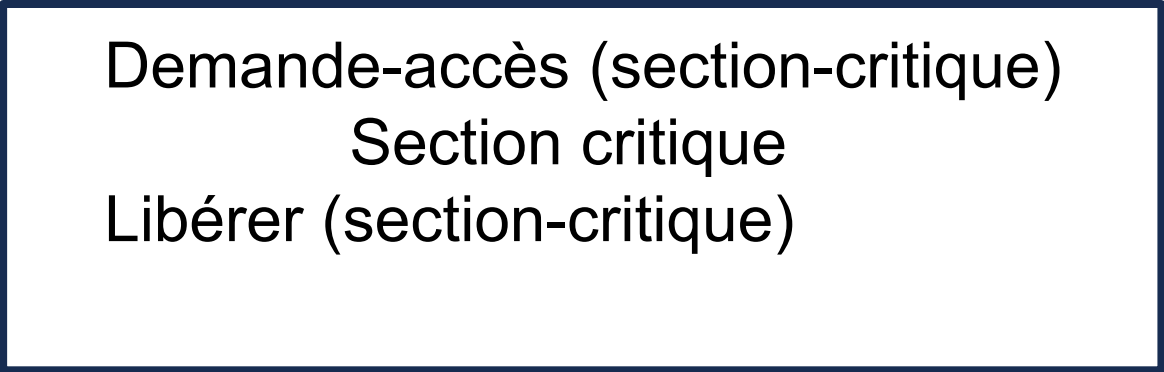
Section Critique

- Quand un processus accède à une donnée partagée, on dit qu'il est dans sa **section critique** (ou région critique)
- Pour éviter les problèmes de concurrence que nous venons d'évoquer, il est nécessaire de s'assurer que si un processus est dans sa section critique tous les autres processus sont exclus de leurs section critique.
- Autrement dit si un processus est dans sa section critique les autres processus continuent à se dérouler en dehors de leurs sections critiques.

Section Critique

Quand un processus sort de sa section critique, il la déclare disponible pour permettre éventuellement à un autre processus d'y accéder.

Ce protocole peut se schématiser par :



Demande-accès (section-critique)
Section critique
Libérer (section-critique)

Section Critique

Pour que des processus parallèles utilisant un objet partagé puissent coopérer, il est nécessaire d'observer les 4 conditions:

- 1. deux processus ne peuvent être en même temps dans leurs Sections Critiques.**
 - 2. Aucune hypothèse n'est faite sur les vitesses des processus**
 - 3. Aucun processus suspendu en dehors d'une section critique ne doit bloquer les autres processus.**
 - 4. Aucun processus ne doit attendre trop longtemps avant d'entrer dans sa SC.**
- L'exécution d'une SC doit être rapide.**

□ Notons qu'être dans une section critique est un statut spécial accordé à un processus. Le processus à l'accès exclusif à une donnée partagée et les autres processus demandant l'accès à cette donnée sont bloqués. De ce fait l'exécution d'une section critique doit être rapide pour minimiser les attentes.

Le Mécanisme d'Exclusion Mutuelle (Locking)

L'objectif est simple : garantir qu'un **seul processus à la fois** se trouve dans la **Section Critique** (la zone du code qui manipule une ressource partagée).

1. Les trois étapes du protocole

Pour sécuriser l'accès, tout processus doit suivre ce rituel :

- **La demande d'entrée (<entrer_SC>)** : Le processus demande la permission. Si quelqu'un est déjà à l'intérieur, il doit attendre.
- **L'exécution (<Section_Critique>)** : Le processus fait son travail (écriture, calcul) en étant seul au monde.
- **La sortie (<Quitter_SC>)** : Le processus signale qu'il a fini pour libérer la place aux autres.

Le Mécanisme d'Exclusion Mutuelle (Locking)

2. Les deux façons d'attendre

A. L'Attente Active (Busy Waiting)

Le concept : Le processus tourne en boucle et vérifie sans arrêt les variables indiquant la présence ou non d'un processus en section critique.

L'analogie : C'est comme quelqu'un qui toque à la porte d'un bureau toutes les 5 secondes pour demander "C'est libre ?".

Inconvénient : Cela gaspille énormément de puissance CPU pour rien.

B. L'Attente Non-Active (Sleep & Wakeup)

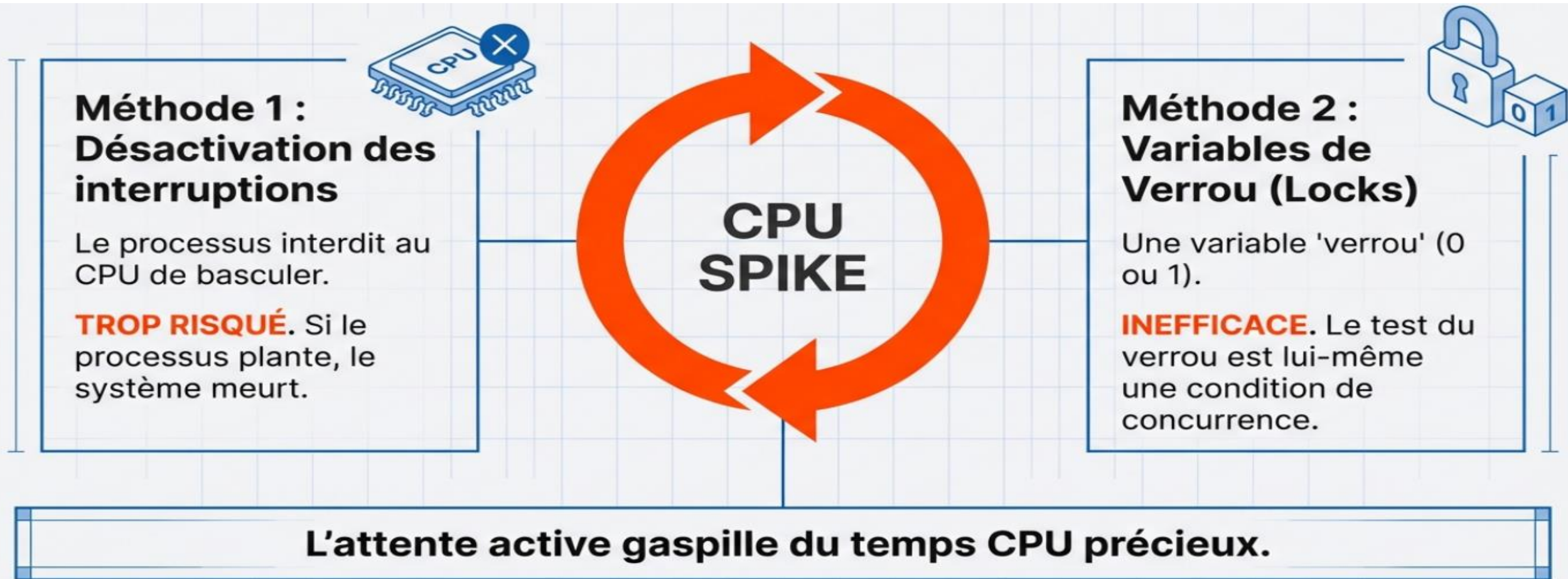
Le concept : Le processus se met dans l'état "endormi" (bloqué) et ne sera « réveillé » que lorsqu'il sera autorisé à entrer en section critique. Le système d'exploitation le retire du processeur.

Le réveil : C'est le processus qui sort de la section critique qui a la responsabilité de "réveiller" celui qui dort.

Avantage : C'est beaucoup plus efficace car le CPU peut travailler sur autre chose pendant l'attente.

Exclusion Mutuelle avec Attente Active

Les solutions de l'exclusion mutuelle par attente active:



Exclusion Mutuelle avec Attente Active

Désactivation des interruptions

Lorsqu'une interruption se produit, le processeur interrompt le programme en cours d'exécution pour exécuter un traitement (le gestionnaire d'interruption). Cela peut intervenir au milieu d'une section critique, provoquant des accès concurrents à des données partagées si le gestionnaire manipule les mêmes ressources.

Solution: **désactiver les interruptions** avant d'entrer dans la section critique, puis à les réactiver à la sortie.

Exemple du code : boucle avec désactivation/réactivation des interruptions

```
while (1) { // un processus qui s'exécute en continu
    < Section Restante >
    Masquer les interruptions // Pendant ce temps, le processeur ne répond à aucune interruption matérielle.
    < Section Critique >
    Démasquer les interruptions
}
```

Exclusion Mutuelle avec Attente Active

Désactivation des interruptions

Limites:

- **Risque de blocage permanent du système:** si une erreur survient pendant l'exécution de la section critique (par exemple un bug, une exception ou un plantage), les interruptions restent masquées. Le processeur ne peut alors plus répondre à aucun événement externe.
- **Retour à un fonctionnement mono-programmé:** pendant toute la durée où les interruptions sont masquées, le mécanisme d'ordonnancement (souvent déclenché par une interruption horloge) est inhibé. Aucun changement de contexte ne peut avoir lieu : le processeur exécute exclusivement le processus courant jusqu'à la réactivation des interruptions. On perd donc le bénéfice du multitâche.
- **Privilège excessif accordé à un processus utilisateur:** la désactivation des interruptions est une instruction privilégiée qui ne devrait être accessible qu'au noyau. Si un processus utilisateur pouvait l'exécuter, il aurait la capacité de monopoliser le processeur et de paralyser tout le système.

Résumé: bien que la désactivation des interruptions soit efficace pour de très courtes sections critiques dans le noyau, elle présente des risques importants qui la rendent inadaptée à une utilisation générale, surtout en contexte multi-programmé et multi-utilisateur.

Exclusion Mutuelle avec Attente Active

Verrous

- Le **verrou** est une variable partagée accessible par tous les processus (ou threads) qui souhaitent utiliser une ressource commune.
- C'est un **objet géré par le système d'exploitation**, utilisé pour contrôler l'accès à une section critique.
- Deux opérations système lui sont associées :
 - ❑ **Verrouiller (lock)** : permet à un processus de prendre le verrou afin d'entrer dans la section critique. Si le verrou est déjà pris, le processus doit attendre.
 - ❑ **Déverrouiller (unlock)** : permet au processus qui détient le verrou de le libérer afin qu'un autre processus puisse y accéder.

Ainsi, le verrou garantit qu'un seul processus à la fois peut accéder à la ressource partagée.

Exclusion Mutuelle avec Attente Active

Verrous

- Considérons le cas de 2 processus p1 et p2.
- Une première approche consiste à associer à une section critique une variable booléenne qui dans l'état vrai interdit l'accès à la section critique, et l'autorise dans l'état faux.
- Ce verrou est une variable partagée entre tous les processus cherchant à accéder à la ressource ainsi protégée.
- Les deux processus p1 et p2 répondent à la description suivante :

Exclusion Mutuelle avec Attente Active

Verrous

```
Var v : SHARED BOOLEEN (: = false)
Parbegin
P1 : begin
Test1 : if v then go to test1 //il attend
      Else v := true // il verrouille
      Section critique // entre dans la section critique
      V := false // il libère
      Endif
      End
P2 : begin
Test2 : if v alors go to test2
      Else v := true
      Section critique
      V :=false
      Endif
      End
Parend.
```

Exclusion Mutuelle avec Attente Active

Verrous

- Les codes des sections critiques de p1 et p2 peuvent ou non être les mêmes. Ce qui importe est que la variable (v) soit partagée par les processus.
- La demande d'accès est réalisée à l'aide de la boucle test : **if v alors go to test** :
 - ✓ si v est dans l'état true cela veut dire qu'un autre processus est dans sa section critique.
 - ✓ L'affectation $v := \text{true}$ permet de condamner l'accès par un autre processus.
 - ✓ La libération est réalisée par l'affectation $v := \text{false}$ permettant à un éventuel processus en attente de sortir de la boucle test : **if v alors go to test. While v do ;**

Exclusion Mutuelle avec Attente Active

Verrous

Cette solution n'assure pas l'exclusion mutuelle.

Où est le problème ?

Imagine :

- Au même moment, P1 teste v
- Au même moment, P2 teste v

$v = \text{false}$

Les deux voient false

Les deux mettent $v := \text{true}$

Les deux entrent dans la section critique ✗

👉 Donc le verrou ne marche pas correctement.

Exclusion Mutuelle avec Attente Active

Verrous

- ❑ Un autre reproche que l'on peut formuler à l'encontre de cette solution est qu'elle réalise une **attente active** (busy form of waiting).
- ❑ Un processus qui attend l'accès à la section critique consomme du temps processeur à vérifier que l'accès est refusé.

Exclusion Mutuelle avec Attente Active

L'alternance de processus

- Le principe de cette méthode est d'imposer une stricte alternance de passage de p_1 et p_2 dans leur section critique.
- On matérialise cette condition par une variable partagée (valant 1 ou 2) indiquant le numéro du processus autorisé à entrer en section critique.
- Un processus aura la politesse de faire en sorte que le prochain processus autorisé soit son partenaire.
- Il faut noter que la méthode est décrite pour deux processus, mais pourrait éventuellement être généralisée.

Exclusion Mutuelle avec Attente Active

L'alternance de processus

Supposons que la valeur initiale de cette variable de passage est 1.

Var passage : shared integer (:= 1)

Parbegin

P1 : begin

Test1 : if passage <> 1 then go to test1 // [while passage <> 1 do];

Else

Section critique

Passage := 2

Endif

P2 begin

Test2 : if passage <> 2 then go to test2

Else

Section critique

Passage := 1

Endif

Parend

Exclusion Mutuelle avec Attente Active

L'alternance de processus

Contrairement à la précédente, cette solution assure l'exclusion mutuelle.

En effet, p_1 ne peut entrer en section critique que si `passage` est égal à 1, cette valeur interdisant à tout autre processus de pénétrer en section critique. Par ailleurs, la valeur de `passage` n'est modifiée qu'une fois la section critique terminée. On n'a donc au plus qu'un processus en section critique.

Le problème lié à cette méthode, outre l'attente active, est qu'elle impose une **stricte alternance** des passages en section critique.

Supposons que p_1 après être sorti de la section critique, se termine. Sa dernière affectation (`passage` : = 2) permettra à p_2 de dérouler une fois cette section critique; mais l'affectation `passage` : = 1 lui interdit tout accès ultérieur.

Un processus bloqué en dehors de sa section critique peut donc empêcher les autres d'y accéder (bloquer).

Exclusion Mutuelle avec Attente Active

Solution de Paterson

La solution de Peterson est une technique d'**attente active** qui permet de synchroniser **deux processus** sans utiliser de mécanismes matériels complexes.

Elle repose sur :

✓ l'intention d'entrer en section critique

✓ la priorité entre les processus

❑ **Variables partagées**

boolean flag[2]; // flag[i] = true si P_i veut entrer

int turn; // indique à qui c'est le tour

❑ **Idée**

👉 Chaque processus dit :

“Je veux entrer” (flag[i] = true)

“Je te laisse la priorité” (turn = j)

Celui qui a la priorité passe, l'autre attend.

Exclusion Mutuelle avec Attente Active

Solution de Paterson (Exemple)

Situation

Deux processus :

P0 veut accéder à une variable x

P1 veut aussi accéder à x

◆ **Cas : P0 et P1 veulent entrer en même temps**

1.P0 exécute :

```
flag[0] = true;
```

```
turn = 1;
```

2.P1 exécute :

```
flag[1] = true;
```

```
turn = 0;
```

◆ Que se passe-t-il ?

P0 vérifie la condition d'attente:

```
while (flag[1] && turn == 1)
```

P1 vérifie la condition d'attente:

```
while (flag[0] && turn == 0)
```

👉 À ce stade, $turn$ vaut 0 (dernière affectation faite par P1).

Pour P0 : $flag[1]$ est vrai, mais $turn == 1$ est faux → la condition du `while` est fausse.

P0 entre en section critique.

Pour P1 : $flag[0]$ est vrai et $turn == 0$ est vrai → la condition du `while` est vraie.

P1 attend.

Ainsi, l'algorithme garantit qu'un seul processus entre en section critique, même s'ils essaient d'y accéder simultanément.

Conclusion

Les approches examinées jusqu'ici reposent toutes sur une forme d'attente active, ce qui nuit aux performances globales du système. Pourtant, un mécanisme de synchronisation bien conçu doit idéalement réunir trois caractéristiques :

- Assurer l'exclusion mutuelle ;
- Garantir qu'un processus bloqué hors de sa section critique ne puisse pas empêcher d'autres processus d'accéder à leurs propres sections critiques ;
- Éviter tout recours à l'attente active.

Dans ce qui va suivre, nous allons explorer comment satisfaire les deux premières exigences tout en éliminant l'attente active.

Exclusion Mutuelle avec Attente Non Active

Principe: désigne un mécanisme où un processus qui ne peut pas entrer dans sa section critique est mis en sommeil (bloqué) par le système d'exploitation. Il ne consomme donc pas de temps CPU en boucle vide. Il sera réveillé uniquement lorsque la ressource deviendra disponible.

Sommeil et activation :

- La première primitive permet de bloquer un processus quand il lui est interdit de rentrer dans sa section critique (**Sleep**)
- La deuxième (**Wakeup**) réveille un process bloqué.

Exclusion Mutuelle avec Attente Non Active

Principe général :

- Un processus qui échoue à obtenir l'accès est mis en sommeil (bloqué) par le système d'exploitation.
- Il ne consomme aucun temps CPU tant qu'il attend.
- Il sera réveillé automatiquement lorsque la ressource devient disponible.

Mécanismes fournis par le système d'exploitation:

Deux appels système permettent cette stratégie :

- SLEEP : met le processus appelant dans l'état « endormi ». Il reste bloqué jusqu'à ce qu'un autre processus le réveille.
- WAKEUP(processus) : réveille un processus spécifique (celui passé en paramètre).

Solutions d'exclusion mutuelle basées sur l'attente non active :

- Les sémaphores
- Les moniteurs

Exclusion Mutuelle avec Attente Non Active

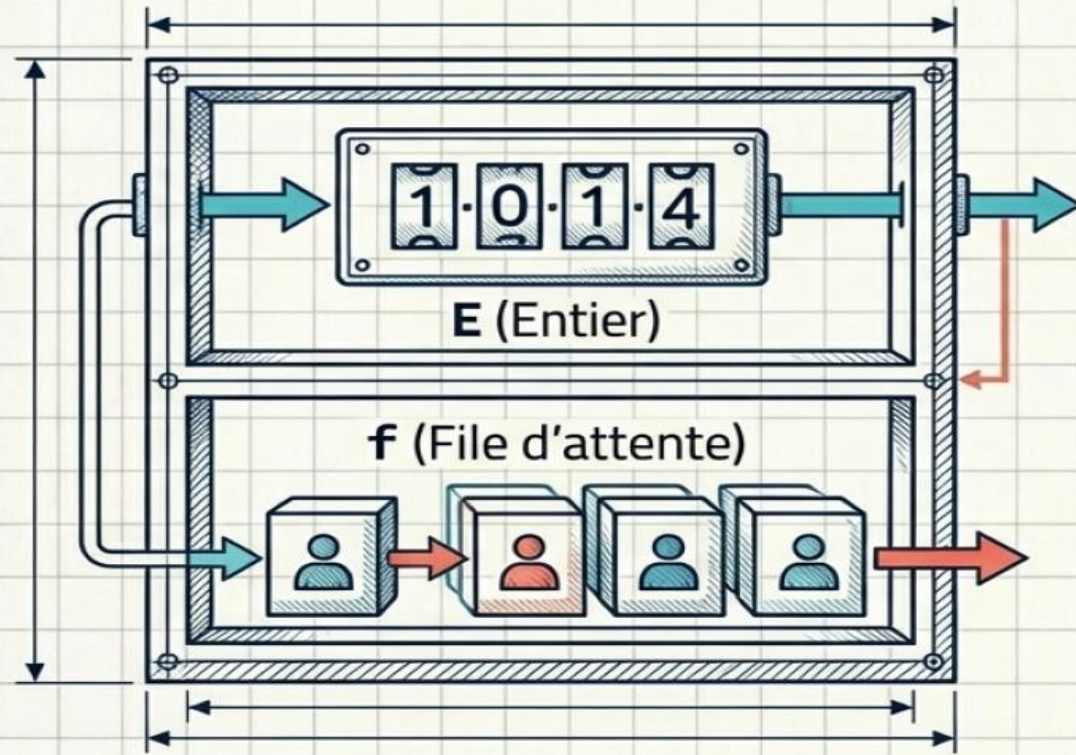
Sémaphore

- Une solution a été introduite par Dijkstra (1965), faisait appel à des entités nommées **sémaphores**.
- Un sémaphore est formé d'une variable entière appelée *compteur* et d'une *file d'attente* de processus.
- Un sémaphore est une extension du verrou classique. Il ne se limite pas à un seul processus, mais permet à plusieurs processus d'accéder simultanément à une ressource.

Exclusion Mutuelle avec Attente Non Active

Sémaphore

Un mécanisme indivisible permettant à un processus de se bloquer ou de réveiller un autre, indépendant des vitesses d'exécution.



WAIT (S) - "Puis-je passer ?"

Si $E \leq 0$:

↳ Le processus est bloqué et mis dans **f**.

Sinon :

↳ **E** est décrémenté.

SIGNAL (S) - 'J'ai terminé.'

E est incrémenté.

Si **f n'est pas vide :**

↳ Extrait un processus de **f** et le réveille.

Exclusion Mutuelle avec Attente Non Active

Sémaphore

Analogie : un distributeur de jetons

- Le sémaphore gère un ensemble de jetons (ou permissions).
- Chaque jeton autorise un processus à utiliser la ressource.
- Le nombre total de jetons est fixe et défini à la création du sémaphore.
- Pour accéder à la ressource, un processus doit emprunter un jeton.
- Après usage, il rend le jeton au sémaphore.

Définition formelle

Un sémaphore est une variable entière positive (ou nulle).
Sa valeur initiale correspond au nombre de jetons disponibles.

Deux opérations atomiques sont possibles :

- P (du néerlandais Proberen = essayer) : demande un jeton → si aucun jeton disponible, le processus attend.
- V (de Verhogen = incrémenter) : rend un jeton → réveille un processus en attente éventuel.

Exclusion Mutuelle avec Attente Non Active

Sémaphore

Une variable de type sémaphore peut être manipulée par trois primitives :

- une primitive d'initialisation,
- une primitive de demande de ressource,
- une primitive de libération de ressource.

La primitive d'initialisation, moins fondamentale que les autres répond à la description suivante :

```
typedef struct semaphore {  
    int E;          // valeur initiale = nombre de jetons  
    File-Attente S.F; // file d'attente des processus bloqués  
} Semaphore;  
void init(Semaphore S, int EO) {  
    S.E = EO;        // initialise la valeur du sémaphore  
    S.F.Tête = NULL; // initialise la tête de file à vide  
    S.F.Queue = NULL; // initialise la queue de file à vide  
}
```

Elle sert à fixer la valeur initiale de l'entier associé au sémaphore (compteur) et à initialiser à 'vide' la file d'attente.

Exclusion Mutuelle avec Attente Non Active

Sémaphore

Primitive P (demander la ressource)

C

```
void P(Semaphore S) {  
    S.E = S.E - 1;      // retirer un jeton  
    if (S.E < 0)         // plus aucun jeton disponible ?  
        Bloquer-Processus(S.F); // bloquer le processus dans la file  
}
```

Primitive V (libérer la ressource)

C

```
void V(Semaphore S) {  
    S.E = S.E + 1;      // rendre un jeton  
    if (S.E <= 0)       // y a-t-il des processus bloqués ?  
        Extraire-Processus(S.F); // réveiller un processus de la file  
}
```

Exclusion Mutuelle avec Attente Non Active

Sémaphore

- ❑ Nous allons maintenant voir comment la notion de sémaphore peut être utilisée pour assurer l'exclusion mutuelle à une ressource, c'est à dire pour protéger une section critique.
- ❑ A chaque ressource en accès exclusif sera associé un sémaphore. La valeur initiale de ce sémaphore doit être telle qu'elle autorise un processus et un seul à exécuter une opération wait sans se bloquer. Cette valeur initiale est donc 1.

```
Var S : SHARED sémaphore (: = 1)
Parabegin
P1 : begin
    Wait (s) //appelle l'opération Wait (équivalent de P)
    Section critique;
    Signal (s) // incrémente S (le remet à 0 ou 1).
End
```

```
P2 : Begin
    Wait (s)
    Section critique
    Signal (s)
End
Paraend
```


Exclusion Mutuelle avec Attente Non Active

Sémaphore

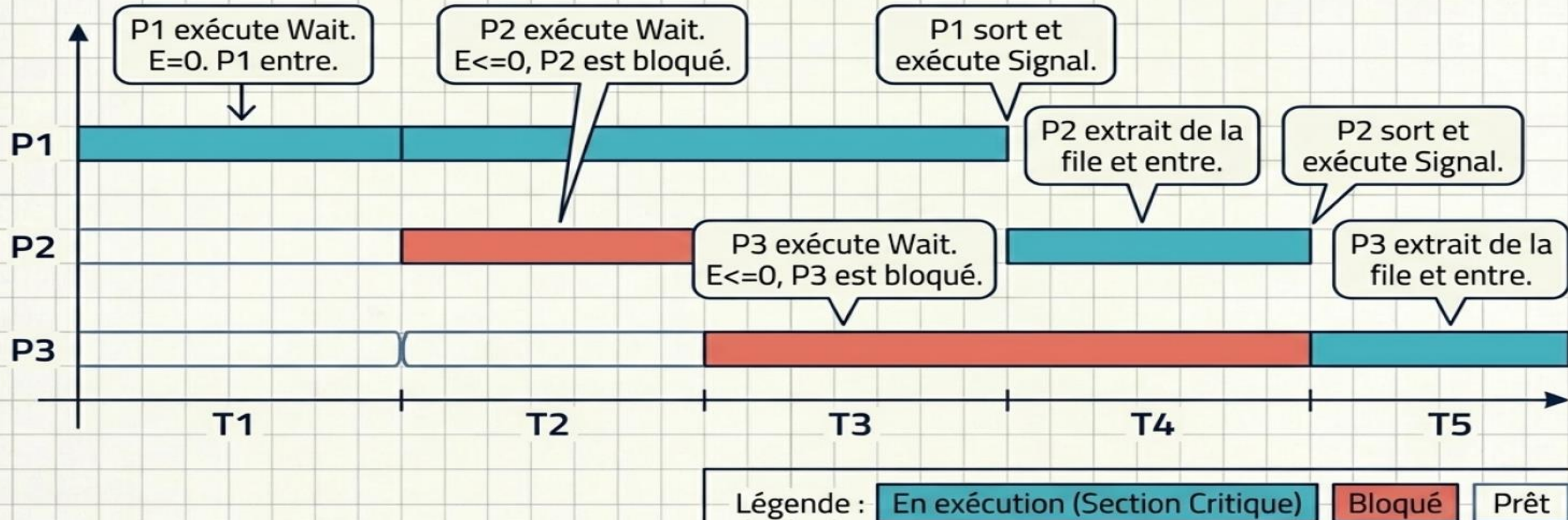
- On peut illustrer un peu plus concrètement le fonctionnement de ce mécanisme en considérant ce qui se passe, avec, par exemple, trois processus en présence (p1, p2, p3), cherchant à accéder à une même ressource.

Événement	S.E	S.F	Commentaires
Init (s,1)	1	Vide	Initialisation de s, processus ancêtre commun.
Wait (s) par p1	0	Vide	P1 exécute wait (s) et entre en section critique. Il y est interrompu et p2 prend le contrôle.
Wait (s) par p2	-1	P2	P2 exécute wait et se fait bloquer. P3 prend le contrôle.
Wait (s) par p3	-2	P2,P3	P3 exécute wait et se bloque. le processeur affecté à P1
Signal (s) p1	-1	p3	P2 est extrait de s.f et mis dans la file des processus prêts. Il est réactivé quelques temps après.
Signal (s) p2	0	Vide	P3 est extrait de s.f et mis dans la file des processus prêts. Il est réactivé quelques temps après.
Signal (s) p3	1	vide	Retour à l'état initial, aucun processus à réactiver.

Exclusion Mutuelle avec Attente Non Active

Sémaphore

Contexte : Sémaphore Mutex = 1. Trois processus (P1, P2, P3).



Exclusion Mutuelle avec Attente Non Active

Sémaphore

Exercice :

On suppose qu'on a 3 processus P1, P2 et P3 qui demandent d'entrer en section critique. Résoudre ce problème avec le moyen des sémaphores.

Instructions	Valeur de S.E	File d'attente S.F	Commentaires
P1 exécute P(S)	0	---	La S.F est vide, P1 arrive en premier à la file. P2 et P3 reste dans la S.F En attendant la libération de la section critique.
P2 exécute P(S)	-1	P2	
P3 exécute P(S)	-2	P3, P2	
Section-Critique_P1 ()			
P1 exécute V(S)	-1	P3	P1 quitte la section critique, P2 entre
Section-Critique_P2 ()			
P2 exécute V(S)	0	---	P2 quitte la section critique, P3 entre
Section-Critique_P3 ()			
P3 exécute V(S)	1	---	P3 quitte la section critique. La S.F redevient vide.

Application au problème du Producteur-Consommateur

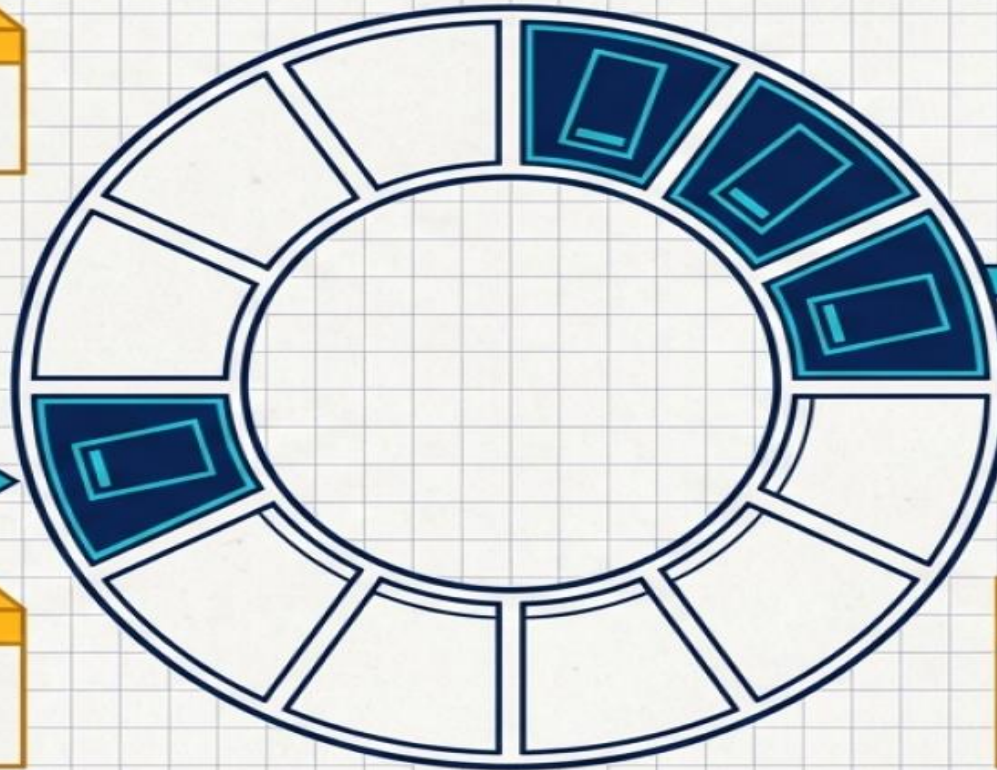
Modèle Classique 1 : Producteur-Consommateur

Deux processus coopèrent pour partager un tampon mémoire circulaire de taille N.

1. Exclusion Mutuelle

L'accès au tampon (insertion ou retrait) doit être strictement exclusif.

Producteur



Consommateur



2. Tampon Plein

Le Producteur doit s'endormir s'il n'y a plus aucune place libre.

3. Tampon Vide

Le Consommateur doit s'endormir s'il n'y a aucun élément à lire.

Stratégie : La solution nécessite la coordination de 3 sémaphores distincts.

Application au problème du Producteur-Consommateur

Solution

Les sémaphores utilisées:

Sémaphore	Valeur initiale	Signification
mutex	1	Protège l'accès exclusif au tampon (section critique).
vide	N (taille du tampon)	Nombre de places libres dans le tampon.
plein	0	Nombre de données présentes (places occupées).

Application au problème du Producteur-Consommateur

Solution

```
semaphore mutex = 1;  
semaphore vide = N;  
semaphore plein = 0;
```

// Producteur

```
void producteur() {  
    int item;  
    while (true) {  
        item = produire();           // fabriquer une donnée  
        P(vide);                     // attendre une place libre  
        P(mutex);                    // verrouiller l'accès au tampon  
        // déposer item dans le tampon  
        V(mutex);                    // libérer le tampon  
        V(plein);                     // signaler une donnée disponible  
    }  
}
```

// Consommateur

```
void consommateur() {  
    int item;  
    while (true) {  
        P(plein);                     // attendre une donnée  
        P(mutex);                     // verrouiller l'accès au tampon  
        // retirer item du tampon  
        V(mutex);                     // libérer le tampon  
        V(vide);                       // signaler une place libre  
        consommer(item);              // utiliser la donnée  
    }  
}
```

Problème rappelé

- Plusieurs lecteurs peuvent lire en même temps.
- Un seul rédacteur peut écrire à la fois.
- Pendant qu'un rédacteur écrit, aucun lecteur ne peut lire.
- Pendant qu'un ou plusieurs lecteurs lisent, aucun rédacteur ne peut écrire.

Application au problème du Producteur-Consommateur

Solution

```
semaphore mutex_l = 1; // protège nb_lecteurs
semaphore mutex_r = 1; // exclusion rédacteurs + bloque premier lecteur
int nb_lecteurs = 0;
```

// Rédacteur

```
void redacteur() {
    while (true) {
        P(&mutex_r); // un seul rédacteur à la fois
        // SECTION CRITIQUE (écriture)
        ecrire_donnees();
        // FIN SECTION CRITIQUE
        V(&mutex_r); // libère pour les autres rédacteurs ou lecteurs
    }
}
```

// Lecteur

```
void lecteur() {
    while (true) {
        P(&mutex_l); // verrouille l'accès à nb_lecteurs
        nb_lecteurs++;
        if (nb_lecteurs == 1) // premier lecteur : bloque les rédacteurs
            P(&mutex_r);
        V(&mutex_l); // libère nb_lecteurs

        // SECTION CRITIQUE (lecture)
        lire_donnees();
        // FIN SECTION CRITIQUE
        P(&mutex_l); // re-verrouille nb_lecteurs
        nb_lecteurs--;
        if (nb_lecteurs == 0) // dernier lecteur : autorise les rédacteurs
            V(&mutex_r);
        V(&mutex_l); // libère nb_lecteurs
    }
}
```


Exclusion Mutuelle avec Attente Non Active

Les moniteurs

Un **moniteur** est une structure qui :

- ❑ regroupe variables + fonctions + synchronisation.
- ❑ garantit qu' un seul processus à la fois entre dans la section critique.

Donc contrairement aux sémaphores :

- ❑ tu n'écris pas $P(S)$ / $V(S)$
- ❑ le système gère automatiquement l'exclusion mutuelle

Idée principale

Un moniteur fonctionne comme une **boîte sécurisée** :

Un seul processus peut entrer dedans 

Les autres attendent automatiquement 

Exclusion Mutuelle avec Attente Non Active

Les moniteurs

Structure d'un moniteur

Un moniteur contient :

- des variables partagées
- des procédures (fonctions)
- des variables de condition

Variables de condition

Elles servent à attendre ou réveiller :

- wait() → le processus attend
- signal() → réveille un processus

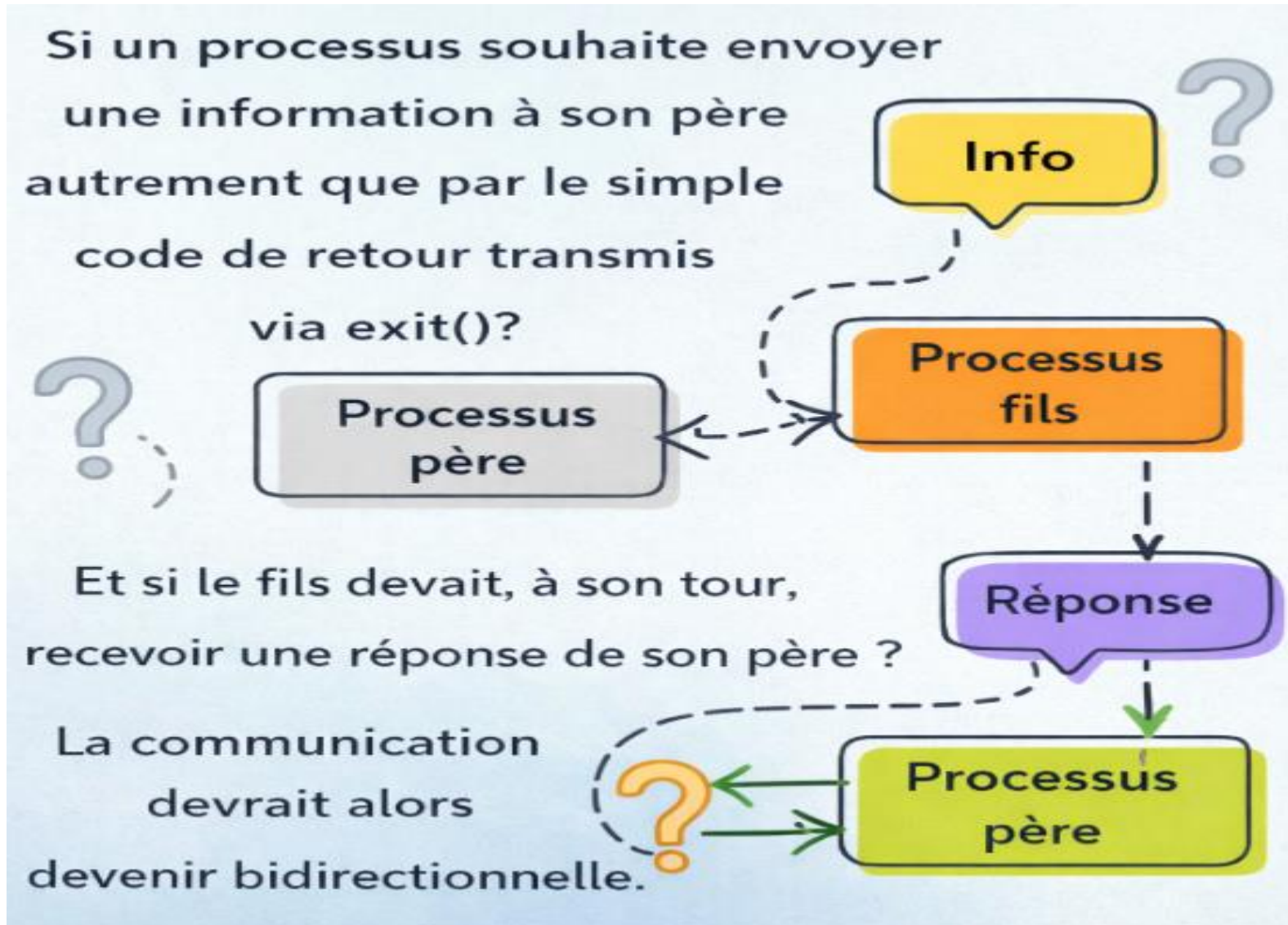
Exclusion Mutuelle avec Attente Non Active

Les moniteurs

Exemple simple

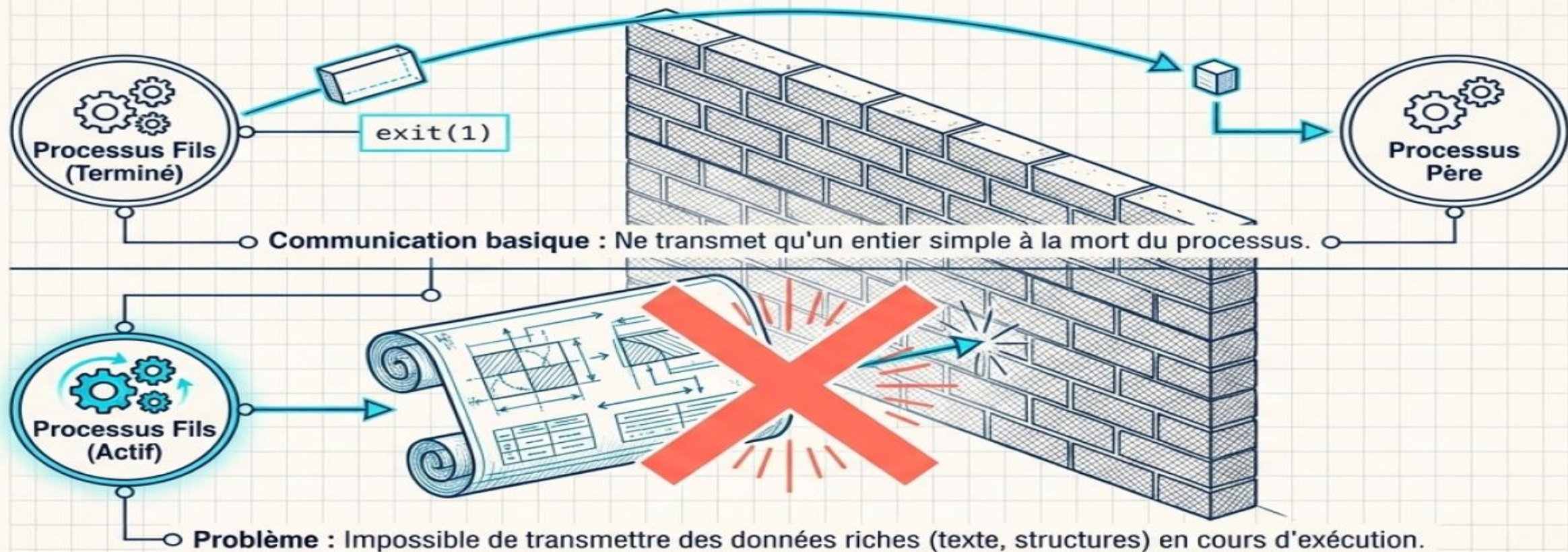
```
monitor Compte {
    int solde;
    Condition soldeSuffisant; // variable de condition
    void deposer(int montant) {
        solde = solde + montant;
        soldeSuffisant.signal(); // réveiller un éventuel thread qui attendait
    }
    void retirer(int montant) {
        while (solde < montant) {
            soldeSuffisant.wait(); // attendre un dépôt
        }
        solde = solde - montant;
        // pas de signal ici (on ne réveille personne après un retrait)
    }
}
```

Mécanismes de communication inter-processus



Mécanismes de communication inter-processus

Au-delà de la Synchronisation : Le besoin d'IPC



La Réponse : Les mécanismes de Communication Inter-Processus (IPC), permettant un échange de données riche, bidirectionnel et à chaud.

Mécanismes de communication inter-processus

Diagnostic : Code de retour vs IPC

Synthèse des capacités de communication au sein du système d'exploitation.

Aspect	Code de retour (exit)	Mécanismes IPC
Type d'information	Entier simple (succès/échec)	Données complexes (messages, structures, fichiers)
Sens de communication	Unidirectionnel (Fils -> Père uniquement)	Bidirectionnel (Apparentés ou indépendants)
Moment de l'échange	À la terminaison (mort du processus)	Pendant l'exécution (à chaud)
Outils typiques	<code>exit()</code> et <code>wait()</code>	Pipes, Files de messages, Sockets, Mémoire partagée

Mécanismes de communication inter-processus

Pourquoi a-t-on besoin de l'IPC ?

Quand un programme est divisé en plusieurs processus (par exemple pour profiter de plusieurs cœurs ou pour isoler des tâches), ces processus doivent parfois **se parler** ou **se partager des données**.

L'IPC est l'ensemble des outils du système d'exploitation qui rendent cette communication possible, que les processus soient parents/enfants ou totalement indépendants.

Avantage par rapport aux fichiers : plutôt que d'écrire des données dans un fichier puis de le relire (lent et source de conflits), l'IPC permet un échange plus rapide et mieux contrôlé.

Mécanismes de communication inter-processus

1. Communication par copie

Principe : Les données sont **dupliquées** du processus émetteur vers le processus récepteur. Chaque processus possède sa propre copie.

Exemples :

- **Tube (pipe)** : unidirectionnel.
- **Files de messages** : bidirectionnel.

Avantage : Simple à utiliser, pas de conflit d'accès car chaque processus travaille sur sa propre copie.

Inconvénient : Moins performant pour de gros volumes de données (à cause des copies répétées).

2. Communication par référence

Principe : Les processus accèdent à une **même zone mémoire partagée**. Il n'y a pas de copie ; ils lisent/écrivent directement au même endroit.

Avantage : Très rapide, idéal pour de grands volumes de données.

Inconvénient : Nécessite une **synchronisation explicite** (sémaphores, mutex, moniteurs) pour éviter les accès concurrents incohérents.

Mécanismes de communication inter-processus par copie

Via la pipe:

Qu'est-ce qu'une pipe ?

Une pipe est comme un **tuyau** dans lequel un processus verse des données et un autre les reçoit à l'autre bout. Les données sortent dans le même ordre qu'elles sont entrées (FIFO).

Règles :

- La communication est à **sens unique** : un seul processus écrit, un seul lit.
- On l'utilise surtout entre un processus père et son fils (par exemple, après un `fork()`).

Avantage : Transmettre des informations sans créer de fichiers temporaires, donc plus rapide et sans risque de conflit sur le système de fichiers.

Mécanismes de communication inter-processus par copie

Pipe dans Linux:

1. Création : l'appel système `pipe(fd)` crée deux descripteurs de fichiers :

`fd[0]` pour lire

`fd[1]` pour écrire

2. Communication :

Le processus écrit avec : `write(fd[1], ...)`

Le processus lit avec `read(fd[0], ...)`

3. Fermeture des extrémités inutiles : chaque processus ferme ce qu'il n'utilise pas pour éviter les blocages.

4. Unidirectionnel et temporaire : le pipe existe uniquement tant que les processus sont actifs.

Mécanismes de communication inter-processus par copie

Pipe dans Linux:

```
int fd[2];
pipe(fd);
if (fork() == 0) {
    // Processus fils (lit)
    close(fd[1]);      // ferme l'écriture – inutile
    read(fd[0], buffer, size);
    close(fd[0]);
} else {
    // Processus père (écrit)
    close(fd[0]);      // ferme la lecture – inutile
    write(fd[1], data, size);
    close(fd[1]);
}
```

Mécanismes de communication inter-processus par copie

Pipe dans Linux:

Fonction	Prototype	Rôle / Utilisation
pipe	<code>int pipe(int fd[2]);</code>	Crée le pipe avec deux descripteurs : fd[0] pour lecture, fd[1] pour écriture
read	<code>ssize_t read(int fd, void *buf, size_t count);</code>	Lire des données depuis le pipe
write	<code>ssize_t write(int fd, const void *buf, size_t count);</code>	Écrire des données dans le pipe
close	<code>int close(int fd);</code>	Fermer une extrémité du pipe